



ECOLE SUPÉRIEURE DE GESTION D'INFORMATIQUE ET DES SCIENCES

RAPPORT DE PROJET AWS WEB SERVICE

THEME

Pipeline d'Ingestion de Données IoT en Temps Réel (Serverless)

Étudiant : **BIKOZI Balakibawi Sylvain**

Identifiant : **sbikozi**

Responsable du cours : **Mofiala Hervé LOKOSSOU**

Région AWS : **eu-west-3 (Paris)**

Sommaire

Sommaire	2
1. Contexte et Objectifs.....	3
2. Architecture Globale	3
2.1 Sous-système d'Ingestion IoT	3
2.2 Sous-système de Documentation	3
2.3 Tableau des Ressources Déployées	3
3. Infrastructure as Code — template.yaml	5
3.1 Déploiement CloudFormation	5
Commande de déploiement :	5
3.2 Capture — Stack CREATE_COMPLETE	5
3.3 Outputs de la pile.....	6
4. Fonction Lambda — index.py.....	7
4.1 Logique de traitement	7
4.2 Code source (index.py)	7
5. Script de Test Client — test_client.py	9
5.1 Payload de test	9
5.2 Payload valide.....	9
5.3 Résultat Test 1 — HTTP 201.....	9
5.4 Résultat Test 2 — Payload corrompu	9
6. Vérification Amazon S3 — Data Lake	10
6.1 Partitionnement des données	10
Commande de vérification :	10
6.2 Capture — Arborescence S3	10
7. Vérification Amazon DynamoDB — Feature Store.....	10
7.1 Structure de la table	10
7.2 Capture — Table DynamoDB avec données insérées	11
8. Supervision — Amazon CloudWatch Logs.....	12
8.1 Groupe de logs.....	12
8.2 Capture — Exécution réussie (Test 1).....	12
8.3 Capture — Exécution en échec (Test 2).....	Erreur ! Signet non défini.
8.4 Comportement lors d'une exception non gérée.....	12
9. Site de Documentation Technique	13
9.1 Upload de la documentation	13
9.2 Validation du blocage des accès directs S3.....	13
9.3 Capture — Accès via CloudFront	13
9.4 Mécanisme de sécurité OAC.....	13
10. Sécurité et Bonnes Pratiques	14
11. Flux de Données — Schéma Récapitulatif.....	15
12. Conclusion	15

1. Contexte et Objectifs

Ce projet s'inscrit dans le cadre du cours *Introduction à AWS* du Master 1 IABD à ESGIS Togo. L'objectif est de concevoir et déployer, de manière entièrement automatisée avec AWS CloudFormation, un pipeline d'ingestion de données IoT **serverless** et hautement disponible.

En tant qu'Ingénieur Big Data & IA pour une entreprise industrielle déployant des milliers de capteurs IoT, la mission est double :

- Mettre en place un point d'entrée HTTP sécurisé, capable de recevoir des flux de données en temps réel.
- Déployer un site web statique hébergeant la documentation technique du projet, accessible uniquement via un CDN sécurisé.

2. Architecture Globale

2.1 Sous-système d'Ingestion IoT

Les capteurs envoient des requêtes **HTTP POST /ingest** vers Amazon CloudFront (distribution d'ingestion). CloudFront transmet la requête à Amazon API Gateway (HTTP v2), qui déclenche la fonction AWS Lambda Python 3.11. La Lambda traite le payload, écrit les données brutes dans Amazon S3 (Data Lake partitionné) et enregistre les métriques agrégées dans Amazon DynamoDB (Feature Store).

2.2 Sous-système de Documentation

Le site de documentation technique est hébergé dans un bucket Amazon S3 privé (*blocage total des accès publics*). Une seconde distribution CloudFront, configurée avec un **Origin Access Control (OAC)**, est le seul chemin autorisé pour accéder au contenu. La politique S3 n'autorise que les requêtes signées par cette distribution.

2.3 Tableau des Ressources Déployées

Ressource AWS	Nom / Identifiant	Rôle
S3 — Data Lake	sbikozi-iot-datalake-629193321657	Stockage brut partitionné (raw-zone)
S3 — Documentation	sbikozi-tech-doc-629193321657	Hébergement site de documentation (privé)
DynamoDB	sbikozi-iot-metrics	Feature Store — métriques agrégées
Lambda	sbikozi-iot-ingestion	Traitement Python 3.11 des données IoT

Ressource AWS	Nom / Identifiant	Rôle
API Gateway HTTP v2	<code>sbikozi-iot-api</code>	Point d'entrée REST POST /ingest
CloudFront — Ingestion	<code>d2nf7bxfx6x6bb.cloudfront.net</code>	CDN devant l'API Gateway (CachingDisabled)
CloudFront — Documentation	<code>d2i1n582pxful4.cloudfront.net</code>	CDN sécurisé devant S3 (OAC, CachingOptimized)
IAM Role	<code>sbikozi-lambda-iot-role</code>	Permissions S3 PutObject + DynamoDB PutItem
CloudFormation Stack	<code>devoir-sbikozi</code>	Infrastructure as Code — eu-west-3

3. Infrastructure as Code template.yaml

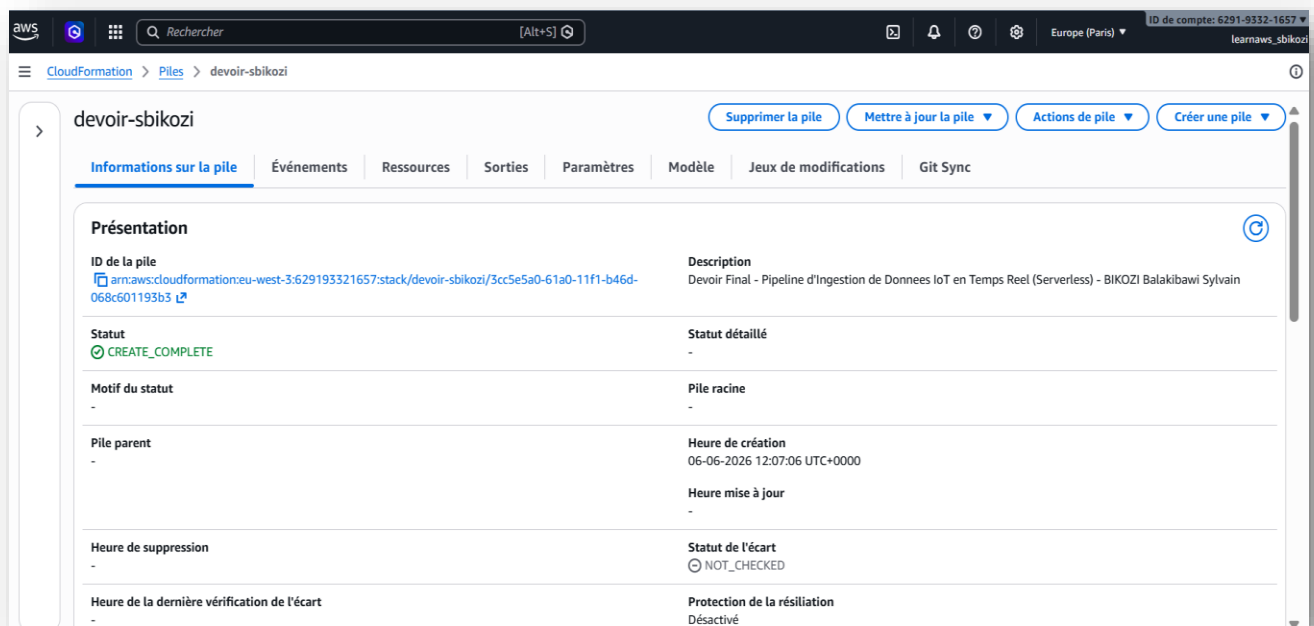
3.1 Déploiement CloudFormation

L'intégralité de l'infrastructure est définie dans un seul fichier `template.yaml` déployé via AWS CLI. Le template utilise des **paramètres conditionnels** (`ShouldCreateRole`, `ShouldCreateOAC`) pour s'adapter aux restrictions IAM des comptes lab (impossibilité de créer un rôle taggé ou un OAC via CloudFormation directement).

Commande de déploiement :

```
aws cloudformation create-stack \  
  --stack-name devoir-sbikozi \  
  --template-body file://infrastructure/template.yaml \  
  --capabilities CAPABILITY_NAMED_IAM \  
  --parameters \  
    ParameterKey=StudentName,ParameterValue=sbikozi \  
  
  ParameterKey=ExistingLambdaRoleArn,ParameterValue=arn:aws:iam::629193321657:role/sbikozi \  
  --lambda-iot-role \  
    ParameterKey=ExistingOACId,ParameterValue=E3A1AYI8AX4F18 \  
  --region eu-west-3
```

3.2 Capture : Stack CREATE_COMPLETE



3.3 Outputs de la pile

Output Key	Valeur
CloudFrontIngestionURL	https://d2nf7bxfxc6xbb.cloudfront.net
CloudFrontDocURL	https://d2i1n582pxful4.cloudfront.net
ApiGatewayDirectURL	https://7pbuflpgr5.execute-api.eu-west-3.amazonaws.com
DataLakeBucketName	sbikozi-iot-datalake-629193321657
TechDocBucketName	sbikozi-tech-doc-629193321657
DynamoDBTableName	sbikozi-iot-metrics
LambdaFunctionName	sbikozi-iot-ingestion

4. Fonction Lambda : index.py

4.1 Logique de traitement

La fonction Lambda Python 3.11 effectue les opérations suivantes à chaque requête **POST /ingest** :

1. **Parsing JSON** : extraction du corps de la requête HTTP via `event['body']`.
2. **Stockage S3** : sauvegarde du payload brut dans le bucket Data Lake avec partitionnement temporel : `raw-zone/year=YYYY/month=MM/<uuid>.json`.
3. **Calcul des métriques** : température moyenne et nombre d'enregistrements en statut **ERROR**.
4. **Écriture DynamoDB** : insertion d'un rapport d'exécution avec : `request_id`, `timestamp`, `s3_path`, `avg_temperature`, `error_count`, `record_count`.
5. **Réponse HTTP 201** : retour du résumé d'ingestion au client.

4.2 Code source (index.py)

```
import json, boto3, os, uuid
from datetime import datetime
from decimal import Decimal

s3 = boto3.client('s3')
ddb = boto3.resource('dynamodb')

def handler(event, context):
    body = json.loads(event.get('body', '{}'))
    records = body.get('records', [])
    if not records:
        return {'statusCode': 400,
                'body': json.dumps({'error': 'No records in payload'})}

    temps = [float(r['temperature']) for r in records if 'temperature' in r]
    avg_temp = round(sum(temps) / len(temps), 2) if temps else 0.0
    error_count = sum(1 for r in records if r.get('status') == 'ERROR')

    now = datetime.utcnow()
    req_id = str(uuid.uuid4())
    s3_key = f"raw-zone/year={now.year}/month={now.month:02d}/{req_id}.json"

    s3.put_object(
        Bucket=os.environ['S3_BUCKET'],
        Key=s3_key,
        Body=json.dumps(body),
        ContentType='application/json'
    )

    ddb.Table(os.environ['DYNAMODB_TABLE']).put_item(Item={
        'request_id': req_id,
        'timestamp': now.isoformat(),
        's3_path': s3_key,
        'avg_temperature': Decimal(str(avg_temp)),
        'error_count': error_count,
        'record_count': len(records)
    })

    return {
        'statusCode': 201,
```

```
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({
            'message': 'Data ingested successfully',
            'request_id': req_id,
            'avg_temperature': avg_temp,
            'error_count': error_count
        })
    }
```

Note technique : Le type `Decimal` est utilisé pour `avg_temperature` car DynamoDB ne supporte pas nativement le type `float` Python avec boto3. Les variables d'environnement `S3_BUCKET` et `DYNAMODB_TABLE` sont injectées par CloudFormation via la section `Environment.Variables` de la fonction Lambda.

5. Script de Test Client : test_client.py

5.1 Payload de test

Le script envoie deux requêtes successives vers <https://d2nf7bxfc6xbb.cloudfront.net/ingest> :

- **Test 1** : Payload valide avec 4 capteurs (2 en statut OK, 2 en statut ERROR). Résultat attendu : **HTTP 201**.
- **Test 2** : Payload corrompu (JSON mal formé). Résultat attendu : **HTTP 500** + Stack Trace dans CloudWatch.

5.2 Payload valide

```
{
  "records": [
    {"sensor_id": "sensor-001", "temperature": 23.5, "status": "OK"},
    {"sensor_id": "sensor-002", "temperature": 41.2, "status": "ERROR"},
    {"sensor_id": "sensor-003", "temperature": 19.8, "status": "OK"},
    {"sensor_id": "sensor-004", "temperature": 55.0, "status": "ERROR"}
  ]
}
```

5.3 Résultat Test 1 : HTTP 201

```
{
  "message": "Data ingested successfully",
  "request_id": "ef4131ad-b2ef-4fb4-965f-0743601b3f1b",
  "avg_temperature": 34.88,
  "error_count": 2
}
```

Résultat : Statut HTTP 201 reçu — ingestion réussie. Température moyenne calculée : **34.88°C** — Anomalies détectées : **2**.

5.4 Résultat Test 2 : Payload corrompu

Statut HTTP : 500
Payload envoyé : "{ ceci est un json invalide !!!"

Résultat attendu : La Lambda lève une `json.JSONDecodeError`. La Stack Trace complète est visible dans Amazon CloudWatch Logs.

6. Vérification Amazon S3 : Data Lake

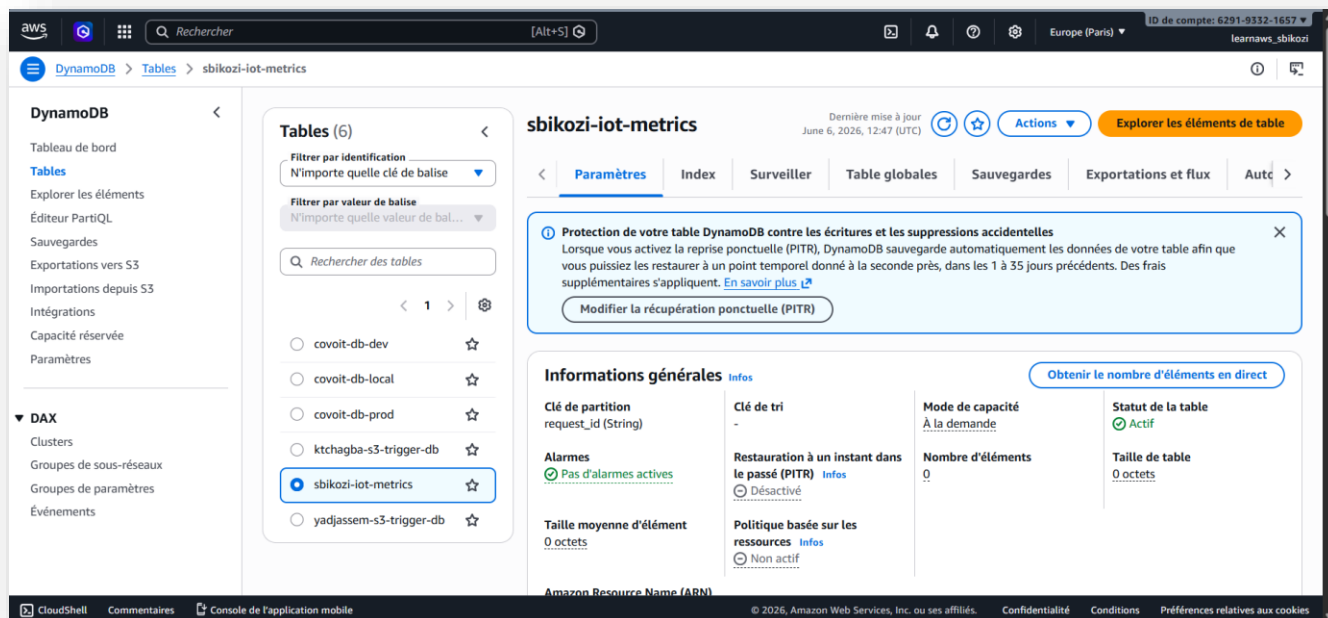
6.1 Partitionnement des données

Les fichiers JSON bruts sont stockés selon le schéma de partitionnement temporel `raw-zone/year=YYYY/month=MM/`. Ce format est compatible avec les moteurs de requêtes analytiques comme Amazon Athena, qui peuvent utiliser ces partitions pour filtrer les données sans scanner l'intégralité du bucket.

Commande de vérification :

```
aws s3 ls s3://sbikozi-iot-datalake-629193321657/raw-zone/year=2026/month=06/ --region eu-west-3
```

6.2 Capture : Arborescence S3



7. Vérification Amazon DynamoDB : Feature Store

7.1 Structure de la table

Attribut	Type	Description
<code>request_id</code>	String (PK)	UUID unique par requête
<code>timestamp</code>	String	Horodatage ISO 8601 UTC
<code>s3_path</code>	String	Chemin du fichier brut dans S3
<code>avg_temperature</code>	Number (Decimal)	Température moyenne calculée
<code>error_count</code>	Number	Nombre d'enregistrements ERROR
<code>record_count</code>	Number	Nombre total d'enregistrements reçus

7.2 Capture : Table DynamoDB avec données insérées

The screenshot shows the Amazon S3 console interface. The breadcrumb navigation at the top indicates the path: `Amazon S3 > Compartiments > sbikozi-iot-datalake-629193321657 > raw-zone/ > year=2026/ > month=06/`. The left sidebar contains the navigation menu with categories: **Amazon S3**, **Compartiments** (with sub-items like 'Compartiments à usage général'), **Fichiers**, **Gestion des accès et sécurité**, and **Gestion du stockage et informations**. The main content area is titled 'month=06/' and has a 'Copier l'URI S3' button. Below the title, there are tabs for 'Objets' and 'Propriétés', with 'Objets' selected. A toolbar above the object list includes buttons for 'Copier l'URI S3', 'Copier l'URL', 'Télécharger', 'Ouvrir', 'Supprimer', 'Actions', 'Créer un dossier', and 'Charger'. A text block explains that objects are fundamental entities stored in Amazon S3 and provides a link to the 'inventaire Amazon S3'. Below this is a search bar and a toggle for 'Afficher les versions'. The object list table has columns: 'Nom', 'Type', 'Dernière modification', 'Taille', and 'Classe de stockage'. It contains one object with the name 'ef4131ad-b2ef-4fb4-965f-0743601b3f1b.json', type 'json', last modified on '06 Jun 2026 12:11:31 PM GMT', size '283.0 o', and storage class 'Standard'.

Amazon S3 > Compartiments > sbikozi-iot-datalake-629193321657 > raw-zone/ > year=2026/ > month=06/

Amazon S3

Compartiments

Compartiments à usage général

Compartiments de répertoires

Compartiments de table

Compartiments de vecteur

Fichiers

Systèmes de fichiers Nouveau

Gestion des accès et sécurité

Points d'accès

Points d'accès pour FSx

Access Grants

Analyseur d'accès IAM

Gestion du stockage et informations

Storage Lens

Opérations par lot

month=06/

Copier l'URI S3

Objets

Propriétés

Objets (1)

Copier l'URI S3 Copier l'URL Télécharger Ouvrir Supprimer Actions Créer un dossier Charger

Les objets sont les entités fondamentales stockées dans Amazon S3. Vous pouvez utiliser l'[inventaire Amazon S3](#) pour obtenir une liste de tous les objets de votre compartiment. Pour que d'autres personnes puissent accéder à vos objets, vous devez leur accorder explicitement des autorisations. [En savoir plus](#)

Rechercher des objets en fonction du préfixe Afficher les versions

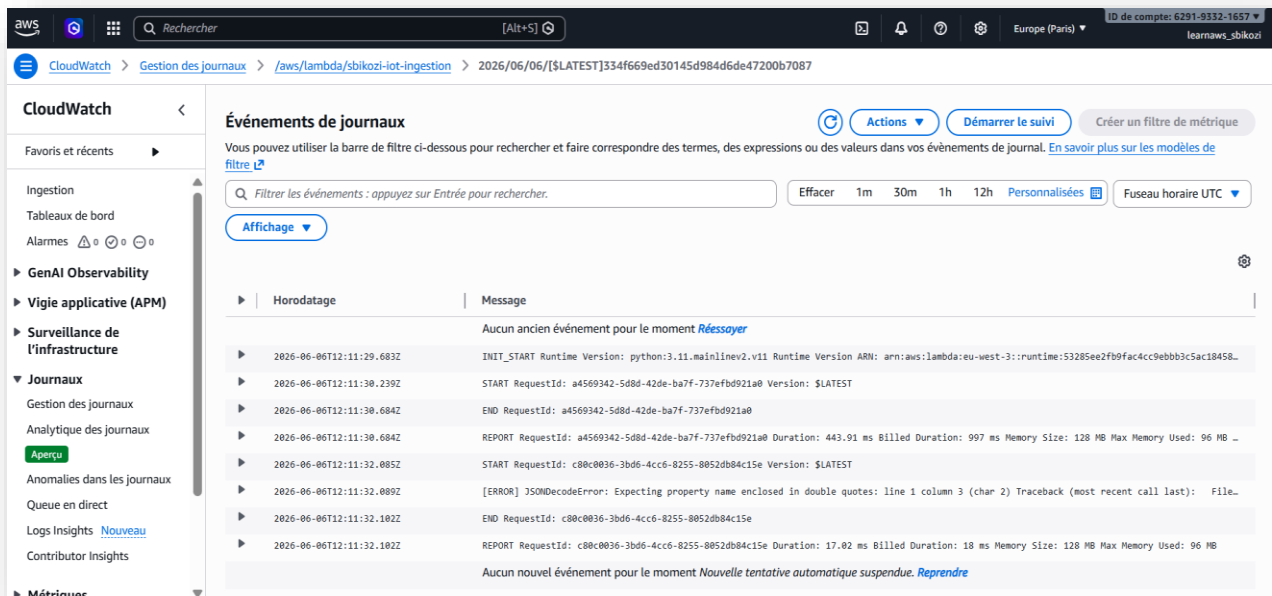
☐	Nom	Type	Dernière modification	Taille	Classe de stockage
☐	ef4131ad-b2ef-4fb4-965f-0743601b3f1b.json	json	06 Jun 2026 12:11:31 PM GMT	283.0 o	Standard

8. Supervision : Amazon CloudWatch Logs

8.1 Groupe de logs

AWS Lambda crée automatiquement un groupe de logs dans Amazon CloudWatch : `/aws/lambda/sbikozi-iot-ingestion`. Chaque invocation génère un log stream contenant les événements **START**, **END**, **REPORT**, ainsi que les sorties Python (**print** et exceptions non gérées).

8.2 Capture : Exécution réussie



8.4 Comportement lors d'une exception non gérée

Lorsque la fonction Lambda lève une exception non gérée (ex. **JSONDecodeError** sur le payload corrompu), AWS Lambda intercepte automatiquement l'erreur et :

- Enregistre la **Stack Trace Python complète** dans CloudWatch Logs.
- Retourne une réponse HTTP **500 Internal Server Error** à l'appelant.
- Incrémente la métrique **Errors** dans CloudWatch Metrics pour la fonction.
- Ne réessaie **pas** automatiquement (contrairement aux invocations asynchrones ou via SQS).

9. Site de Documentation Technique

9.1 Upload de la documentation

Le fichier `index.html` est déposé dans le bucket S3 de documentation via la commande AWS CLI suivante :

```
aws s3 cp index.html s3://sbikozi-tech-doc-629193321657/ --region eu-west-3
```

9.2 Validation du blocage des accès directs S3

L'accès direct à l'URL S3 (`https://sbikozi-tech-doc-629193321657.s3.eu-west-3.amazonaws.com/index.html`) retourne une erreur **Access Denied**, prouvant que le blocage des accès publics est correctement activé sur le bucket.

9.3 Capture : Accès via CloudFront

Pipeline d'Ingestion de Données IoT en Temps Réel (Serverless)

Introduction à AWS — Master 1 IABD — Responsable : Mofiala Hervé LOKOSSOU

BIKOZI Balakibawi Sylvain — 2026

Contexte du Projet

Ce projet met en place une architecture **serverless hautement disponible** pour ingérer en temps réel les flux de données envoyés par des milliers de capteurs IoT industriels via des requêtes HTTP POST. En parallèle, un site de documentation est accessible de manière sécurisée via Amazon CloudFront et un bucket S3 privé avec Origin Access Control (OAC).

Toute l'infrastructure est automatisée avec **AWS CloudFormation** (< template.yaml >).

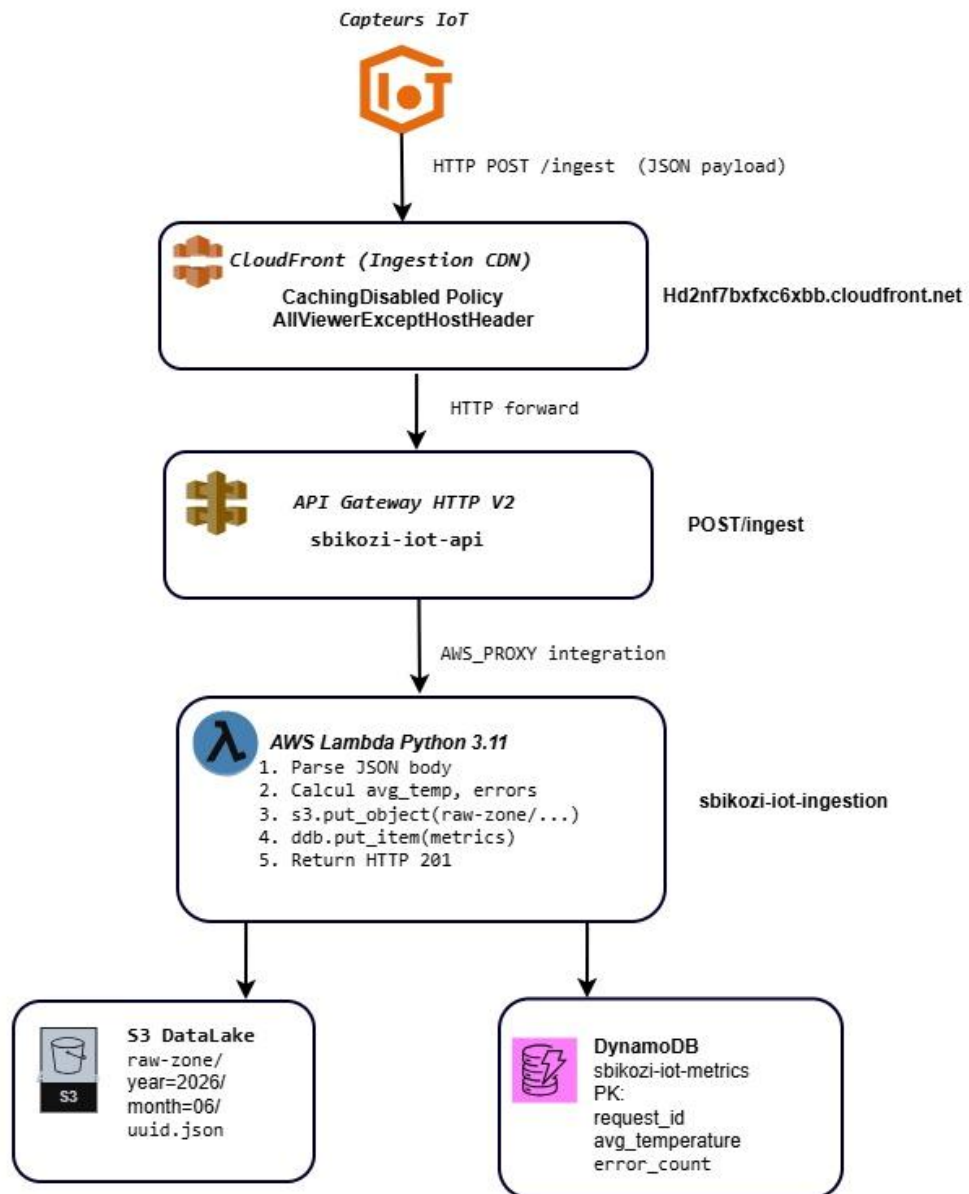
9.4 Mécanisme de sécurité OAC

La politique S3 du bucket de documentation n'autorise que les requêtes signées par la distribution CloudFront (**AWS:SourceArn**). L'OAC signe chaque requête CloudFront → S3 avec SigV4, garantissant que seule la distribution autorisée peut lire le contenu.

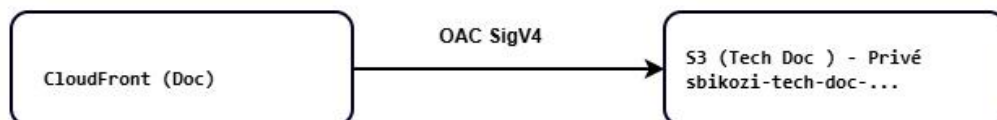
10. Sécurité et Bonnes Pratiques

- **Blocage des accès publics S3** activé sur les deux buckets (`BlockPublicAcls`, `BlockPublicPolicy`, `IgnorePublicAcls`, `RestrictPublicBuckets`).
- **Principe du moindre privilège IAM** : le rôle Lambda n'a que les droits `s3:PutObject` et `dynamodb:PutItem`, limités aux ressources du projet.
- **HTTPS forcé** sur CloudFront (`ViewerProtocolPolicy: redirect-to-https`) et sur l'origine API Gateway (`OriginProtocolPolicy: https-only`).
- **Variables d'environnement** pour `S3_BUCKET` et `DYNAMODB_TABLE` : aucun nom de ressource codé en dur dans le code Lambda.
- **DynamoDB PAY_PER_REQUEST** : pas de capacité provisionnée inutilisée, coût aligné sur l'usage réel.
- **Versioning S3** activé sur le Data Lake pour protection contre les suppressions accidentelles.

11. Flux de Données : Schéma Récapitulatif



Documentation:



12. Conclusion

L'architecture serverless déployée sur AWS répond à l'ensemble des exigences du projet : ingestion en temps réel, stockage structuré en Data Lake, calcul de métriques, sécurisation des accès et documentation accessible via CDN.

L'utilisation de CloudFormation comme unique source de vérité garantit la reproductibilité complète de l'infrastructure : un seul fichier `template.yaml` suffit à recréer l'intégralité des ressources dans n'importe quel compte AWS, dans la région eu-west-3 (Paris).

Les mécanismes de sécurité (OAC, HTTPS forcé, blocage S3, moindre privilège IAM) assurent que ni les données IoT ni la documentation interne ne sont exposés publiquement de manière non contrôlée. CloudWatch Logs garantit la traçabilité complète de chaque exécution Lambda, facilitant le débogage en production.